ROADMAP DEFINITIVO

Desarrollador Android Nativo Senior con Kotlin

De docente a Android Developer freelance — un plan de 12 a 24 meses

Preparado como mentoría permanente — julio 2026

Cómo usar este roadmap

Este documento es tu mapa definitivo, pero un mapa no camina por ti. Está diseñado bajo el principio de Pareto (80/20), aprendizaje activo, práctica deliberada, repetición espaciada y aprendizaje basado en proyectos. Cada nivel tiene el mismo formato: objetivo, conocimientos, recursos, orden, tiempo, errores comunes, proyecto obligatorio, desafío y checklist de dominio. No avances de nivel sin completar el checklist — no como castigo, sino porque cada nivel construye sobre el anterior de forma literal en el mismo proyecto ("Diario de clase"), que evoluciona contigo de la etapa 4 a la etapa 14.

Contenido

- Filosofía de mentoría: a partir de ahora, cada vez que termines una etapa, tráeme tu código y tu checklist. Te voy a evaluar con exigencia real, revisar tu código, hacerte preguntas técnicas, plantearte entrevistas simuladas, y no vas a avanzar hasta demostrar dominio real — no completitud de ejercicios.
- Cuando existan varias formas de hacer algo (ej: MVVM vs MVI, Retrofit vs Ktor), este documento te da la recomendada actualmente por Google y las alternativas, con contexto de cuándo usar cada una — la ingeniería real es sobre trade-offs, no sobre una única 'forma correcta'.
- Las tecnologías referenciadas son las versiones estables actuales del ecosistema Android/Kotlin. Cuando algo está en declive (como Fragments frente a Compose Navigation, o SharedPreferences frente a DataStore) se indica explícitamente para que sepas cuándo aprenderlo solo por mantenimiento de proyectos legacy, no como base de proyectos nuevos.

Nivel 0 — Preparación mental, herramientas y mentalidad de ingeniero

Objetivo

Antes de escribir una sola línea de código necesitas dos cosas: el entorno de trabajo correcto y la mentalidad correcta. Esta etapa existe porque la razón número uno por la que la gente abandona no es la dificultad técnica, es la falta de sistema. Tú ya viviste esto con Flutter: el problema no fue tu capacidad, fue la ausencia de una rutina sostenible y de checkpoints claros de progreso.

Conocimientos (todo lo que cubre esta etapa)

- Instalación y configuración de Android Studio (canal estable, no Beta ni Canary para producción de aprendizaje)
- Estructura de un proyecto Android: manifest, res, build.gradle.kts, módulos
- Gradle: qué es, por qué existe, diferencia entre Groovy DSL y Kotlin DSL (usarás siempre Kotlin DSL — kts)
- Android Virtual Device (AVD) vs dispositivo físico — configurar ambos, activar depuración USB
- ADB (Android Debug Bridge): comandos básicos, logcat, instalación manual de APKs
- Git y GitHub: cuenta, SSH keys, primer repositorio
- Configuración de un flujo de trabajo diario: dónde tomas notas (Notion/Obsidian), dónde guardas código (GitHub), dónde trackeas progreso
- Principio de aprendizaje activo vs pasivo: por qué ver videos NO es aprender
- Técnica Feynman aplicada a programación: explicar cada concepto en tus propias palabras antes de avanzar
- Concepto de 'deuda de aprendizaje': no avanzar sin entender es peor que avanzar lento

Recursos gratuitos

- Android Studio (developer.android.com/studio) — documentación oficial de instalación
- Git — documentación oficial (git-scm.com) y 'Pro Git' (libro gratuito completo)
- GitHub Skills (skills.github.com) — cursos interactivos gratuitos de Git/GitHub
- Android Developers YouTube — playlist 'Android Studio Tips'

Recursos de pago (solo los que realmente valen la pena)

- Ninguno necesario en esta etapa. Cualquier curso de pago aquí es dinero desperdiciado — todo lo de este nivel es 100% cubierto por documentación oficial gratuita.

Orden exacto de estudio

- 1) Instala Android Studio y crea tu primer proyecto vacío (Empty Activity)
- 2) Configura un emulador Pixel con la última API estable
- 3) Aprende Git básico y sube ese proyecto vacío a GitHub
- 4) Diseña tu sistema de seguimiento de progreso (una hoja de cálculo o Notion con las 15 etapas de este roadmap)
- 5) Define tu horario semanal fijo (ver sección de Productividad más abajo) ANTES de tocar Kotlin

Tiempo estimado: 3 a 5 días. No más. Esta etapa es logística, no de aprendizaje profundo — si te tardas más de una semana, el problema es procrastinación disfrazada de preparación.

Errores comunes que debes evitar

- Perder semanas 'decidiendo' entre Android Studio y otros IDEs — no hay otra opción real, Android Studio es el estándar
- Instalar demasiados plugins/temas antes de escribir código
- Empezar sin definir un horario semanal fijo — esto es lo que más predice el abandono
- Comprar cursos antes de saber si el problema es de recursos o de disciplina

Proyecto obligatorio

No hay proyecto de código en este nivel. El 'proyecto' es tu sistema de trabajo: entorno instalado y verificado, repositorio GitHub creado, y un documento de una página con tu horario semanal comprometido.

Desafío (después del proyecto)

Escribe en un documento (te lo voy a pedir cuando avances al Nivel 1) por qué abandonaste Flutter y qué vas a hacer diferente esta vez. Sin esta reflexión honesta, no avanzamos.

Checklist — cómo saber que dominaste esta etapa

- ☐ Android Studio instalado, proyecto Empty Activity corriendo en emulador y en dispositivo físico
- ☐ Puedes explicar qué hace Gradle sin buscarlo
- ☐ Tienes un repositorio en GitHub con tu primer commit
- ☐ Tienes un horario semanal escrito y realista dado tu trabajo docente

Nivel 1 — Fundamentos de programación

Objetivo

Si Kotlin es tu primer lenguaje de programación real (más allá de scripts sueltos), necesitas los conceptos universales antes de la sintaxis específica. Si ya programaste antes (Flutter/Dart cuenta), esta etapa será un repaso rápido pero NO te lo saltes — Dart y Kotlin comparten familia pero difieren en detalles que generan malos hábitos si asumes que 'ya sabes esto'.

Conocimientos (todo lo que cubre esta etapa)

- Variables, tipos de datos, mutabilidad vs inmutabilidad (concepto central en Kotlin: val vs var)
- Operadores aritméticos, lógicos, de comparación
- Estructuras de control: if/else, when (el switch de Kotlin, mucho más poderoso)
- Bucles: for, while, do-while, y su costo real en rendimiento
- Funciones: parámetros, retorno, funciones puras vs funciones con efectos secundarios
- Scope y ciclo de vida de variables
- Manejo básico de errores: try/catch/finally, por qué existen las excepciones
- Entrada/salida básica por consola (para practicar lógica antes de UI)
- Complejidad de código: por qué un código que 'funciona' no siempre es un buen código

Recursos gratuitos

- Kotlin Koans (oficial de JetBrains, play.kotlinlang.org/koans) — ejercicios interactivos gratuitos, el mejor punto de entrada
- Kotlin Docs oficiales — sección 'Basics' (kotlinlang.org/docs/basic-syntax.html)
- freeCodeCamp — curso de Kotlin en YouTube (gratuito, ~4 horas, buen refuerzo)
- Exercism.org — track de Kotlin, ejercicios con mentoría por texto gratuita

Recursos de pago (solo los que realmente valen la pena)

- No es necesario pagar nada en este nivel. Si quieres invertir dinero en el roadmap completo, guárdalo para el Nivel 4-5 (Android SDK/Compose) donde sí hay cursos que aceleran significativamente.

Orden exacto de estudio

- 1) Kotlin Koans completo (todas las categorías 'Introduction' y 'Conventions')
- 2) Ejercicios diarios cortos en Exercism (15-20 min, no maratones)
- 3) Reescribe 3 pequeños programas de consola: calculadora, conversor de temperaturas, validador de contraseñas

Tiempo estimado: 2 a 3 semanas a tu ritmo (recuerda: tienes jornada docente de 6h + familia). Si ya programaste en Dart, puede comprimirse a 1 semana enfocada solo en las diferencias.

Errores comunes que debes evitar

- Copiar y pegar soluciones de Koans sin escribirlas a mano
- Saltarse 'when' porque 'ya conozco switch' — when tiene branches por rangos, tipos y condiciones que switch no tiene

- No practicar manejo de null desde el día 1 (Kotlin obliga a pensar en nulabilidad, es una ventaja frente a Dart/Java, no un obstáculo)

Proyecto obligatorio

CLI (línea de comandos) de 'Gestor de gastos personales': el usuario ingresa gastos, categorías, y el programa calcula totales por categoría, promedio diario, y alerta si se supera un presupuesto. Debe usar funciones separadas, manejo de errores, y al menos un uso correcto de 'when'.

Desafío (después del proyecto)

Añade persistencia simple del gestor de gastos guardando/leyendo un archivo de texto plano (sin bases de datos todavía) usando kotlin.io.

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes explicar la diferencia entre val y var sin dudar
- ☐ Puedes escribir una función que use when con múltiples tipos de condición
- ☐ Tu proyecto del gestor de gastos corre sin errores y maneja entradas inválidas sin crashear

Nivel 2 — Kotlin (núcleo del lenguaje)

Objetivo

Aquí es donde la mayoría de bootcamps se quedan cortos: enseñan sintaxis pero no idioms. Un desarrollador Mid escribe Kotlin que funciona. Un Senior escribe Kotlin que es idiomático — código que cualquier otro Kotlin developer reconoce como 'natural' del lenguaje, no como Java traducido.

Conocimientos (todo lo que cubre esta etapa)

- Null safety profundo: ?, !!, ?:, let/also/apply/run/with (scope functions — cuándo usar cada una, no memorizar, entender)
- Data classes: equals/hashCode/copy generados, cuándo usarlas y cuándo NO (no todo es un data class)
- Sealed classes e interfaces: modelado de estados finitos (esencial para UI state en Compose más adelante)
- Extension functions: cómo y por qué extender clases que no puedes modificar
- Higher-order functions y lambdas: map, filter, reduce, fold — programación funcional aplicada
- Collections: List vs MutableList, Sequence vs List (evaluación perezosa vs ansiosa, impacto real en rendimiento)
- Generics y variance: in/out, por qué existen (covarianza y contravarianza)
- Delegation: by lazy, by Delegates.observable, delegación de interfaces
- Inline functions y reified: por qué inline reduce overhead, qué es reified y cuándo es indispensable
- Coroutines (introducción): suspend functions, CoroutineScope, Dispatchers — el modelo de concurrencia que reemplaza callbacks y RxJava en Android moderno
- Flow: cold streams, StateFlow vs SharedFlow (fundamental para arquitectura moderna)
- Anotaciones y reflection básica: para qué sirven (Room, Hilt las usan intensamente aunque tú no las escribas directamente al inicio)
- DSLs en Kotlin: cómo Kotlin permite crear mini-lenguajes (Compose es un DSL, Gradle Kotlin DSL es un DSL)

Recursos gratuitos

- Kotlin Docs oficiales — secciones avanzadas completas (coroutines, generics, delegation)
- 'Kotlin Coroutines by Tutorials' — capítulos gratuitos disponibles en raywenderlich.com/kotlin
- Roman Elizarov (líder del equipo de Coroutines en JetBrains) — charlas en YouTube sobre coroutines internals
- Philipp Lackner (YouTube) — canal gratuito enfocado 100% en Kotlin/Android moderno, altamente recomendado para tu nivel de entrada

Recursos de pago (solo los que realmente valen la pena)

- 'Kotlin Coroutines and Flow' — Udemy (Philipp Lackner o Denis Panjuta) — vale la pena SOLO si en la práctica sientes que coroutines no te queda claro tras los recursos gratuitos; no lo compres preventivamente
- JetBrains Academy (ahora 'Hyperskill') — track de Kotlin, tiene un plan gratuito limitado y uno de pago; el de pago solo se justifica si prefieres aprendizaje 100% guiado paso a paso

Orden exacto de estudio

- 1) Null safety y scope functions (base de todo lo demás)
- 2) Data classes y sealed classes
- 3) Collections y programación funcional (map/filter/reduce)

- 4) Extension functions y generics
- 5) Coroutines básicas (suspend, launch, async/await)
- 6) Flow básico (solo lo suficiente para entender StateFlow — lo profundizarás en Nivel 6)

Tiempo estimado: 5 a 7 semanas. Este es el nivel más denso de todo el roadmap fuera de arquitectura. No lo comprimas.

Errores comunes que debes evitar

- Usar !! en vez de manejar el null correctamente ('el atajo que se vuelve crash en producción')
- Abusar de scope functions sin entender la diferencia entre 'this' y 'it' en cada una
- Tratar coroutines como 'hilos con otro nombre' — no lo son, son un modelo de concurrencia cooperativa distinto
- Aprender sintaxis de Flow sin entender por qué reemplaza a callbacks/LiveData en arquitectura moderna

Proyecto obligatorio

Librería de utilidades en Kotlin puro (sin Android todavía): un motor de validación de formularios reutilizable usando sealed classes para representar resultados (Success/Error con tipos específicos), extension functions para validaciones comunes (email, teléfono, contraseña fuerte), y funciones de orden superior para componer validadores.

Desafío (después del proyecto)

Convierte el motor de validación para que use coroutines y pueda ejecutar validaciones asíncronas (ej: verificar si un email ya existe consultando una API simulada con delay()).

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes explicar la diferencia entre let, also, apply, run y with sin consultar documentación
- ☐ Puedes modelar un estado de UI (loading/success/error) usando sealed class sin ayuda
- ☐ Entiendes la diferencia entre suspend function y función normal, y por qué una suspend function no bloquea el hilo
- ☐ Puedes explicar qué es StateFlow en tus propias palabras (técnica Feynman) a alguien sin conocimiento técnico

Nivel 3 — Programación Orientada a Objetos y principios de diseño

Objetivo

Kotlin es multiparadigma, pero Android SDK y la mayoría de arquitecturas profesionales están construidas sobre OOP sólido. Sin esta base, tu código 'funciona' pero no escala ni se puede mantener en equipo — que es exactamente lo que las empresas evalúan.

Conocimientos (todo lo que cubre esta etapa)

- Clases, objetos, constructores primarios y secundarios, init blocks
- Herencia: open, override, super — por qué Kotlin hace todo 'final' por defecto (a diferencia de Java)
- Interfaces vs clases abstractas: cuándo usar cada una
- Polimorfismo real (no solo la definición de examen): casos de uso concretos en Android
- Encapsulamiento: visibility modifiers (private, internal, protected, public) y su propósito real en módulos
- Composición sobre herencia: por qué la industria moderna prefiere composición (esencial para entender Jetpack Compose más adelante)
- Principios SOLID aplicados con ejemplos reales de Android (no ejemplos de 'Animal y Perro' — ejemplos de ViewModel, Repository, UseCase)
- Object (singleton) y companion object: diferencias y casos de uso correctos
- Enum classes vs sealed classes: cuándo cada una

Recursos gratuitos

- Kotlin Docs — sección 'Classes and Objects'
- 'Clean Code' de Robert C. Martin — resúmenes y capítulos clave disponibles gratis en múltiples repositorios GitHub de apuntes (busca el libro en tu biblioteca si puedes acceder legalmente, es la referencia histórica aunque los ejemplos son en Java)
- Refactoring Guru (refactoring.guru) — explicación gratuita y visual de SOLID y patrones de diseño, con ejemplos claros

Recursos de pago (solo los que realmente valen la pena)

- 'Clean Code' físico o Kindle si quieres profundizar más allá de resúmenes — inversión única que te sirve para toda tu carrera, no solo Android

Orden exacto de estudio

- 1) Clases, herencia, interfaces
- 2) Encapsulamiento y visibility modifiers
- 3) SOLID (uno por uno, con ejemplo práctico de cada principio)
- 4) Composición sobre herencia

Tiempo estimado: 2 a 3 semanas.

Errores comunes que debes evitar

- Aprender SOLID como teoría sin aplicarlo a código real — se olvida en una semana si no lo practicas
- Abusar de herencia cuando composición resuelve el problema de forma más flexible

- Confundir object (Kotlin) con 'objeto' en el sentido genérico de OOP

Proyecto obligatorio

Sistema de gestión de una biblioteca escolar en consola (conectado a tu contexto docente): clases para Libro, Estudiante, Préstamo, aplicando herencia donde tenga sentido (ej: distintos tipos de usuario: Estudiante/Docente con distintos límites de préstamo), interfaces para comportamientos compartidos, y al menos 3 principios SOLID aplicados explícitamente y documentados en comentarios explicando la decisión.

Desafío (después del proyecto)

Refactoriza el proyecto anterior (el gestor de gastos del Nivel 1) aplicando todo lo aprendido aquí: sepáralo en clases con responsabilidad única, interfaces para las fuentes de datos, y composición en vez de herencia donde corresponda.

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes explicar cada letra de SOLID con un ejemplo propio, no memorizado
- ☐ Puedes justificar por escrito por qué usaste herencia o composición en una decisión de diseño concreta
- ☐ Tu código de la biblioteca escolar no tiene una sola clase con más de una responsabilidad clara

Nivel 4 — Android SDK y fundamentos de la plataforma

Objetivo

Aquí empieza Android 'de verdad'. El objetivo es entender cómo el sistema operativo gestiona tu aplicación — ciclo de vida, procesos, memoria — porque el 80% de los bugs de producción en apps reales vienen de no entender esto, no de errores de sintaxis.

Conocimientos (todo lo que cubre esta etapa)

- Componentes de Android: Activity, Fragment (en declive pero necesario entenderlo — ver nota histórica abajo), Service, BroadcastReceiver, ContentProvider
- AndroidManifest.xml: permisos, declaración de componentes, intent-filters
- Ciclo de vida de Activity (onCreate, onStart, onResume, onPause, onStop, onDestroy) y por qué importa para evitar memory leaks
- Intents explícitos e implícitos, Intent extras, deep links básicos
- Contexto (Context): Application context vs Activity context, por qué usar el incorrecto causa memory leaks
- Recursos (res/): strings.xml, dimens.xml, colors.xml, gestión de internacionalización desde el día 1
- Configuración de dispositivos: qualifiers (valores-es, valores-night, sw600dp) para distintos idiomas, temas y tamaños de pantalla
- Procesos y memoria: cómo Android mata procesos en segundo plano, onSaveInstanceState
- Permisos en tiempo de ejecución (runtime permissions) — modelo desde Android 6.0 en adelante
- ANRs (Application Not Responding): qué los causa y cómo evitarlos (bloquear el hilo principal)

Recursos gratuitos

- Android Developers — 'Fundamentals' (developer.android.com/guide/components/fundamentals)
- Android Developers Codelabs — 'Android Basics with Compose' (codelabs oficiales, gratuitos, con ejercicios guiados)
- Philipp Lackner (YouTube) — serie de fundamentos de Android
- CommonsWare 'Elements of Android' — (algunos capítulos gratuitos en su blog, el resto de pago) referencia técnica muy respetada

Recursos de pago (solo los que realmente valen la pena)

- 'Android Developer Fundamentals' — si prefieres estructura tipo curso sobre documentación libre; opcional, la documentación oficial hoy es excelente y gratuita

Orden exacto de estudio

- 1) Manifest y componentes básicos
- 2) Ciclo de vida de Activity (con logs propios para verlo en tiempo real, no memorizado de un diagrama)
- 3) Recursos e internacionalización
- 4) Intents y navegación básica entre pantallas
- 5) Permisos en runtime

Tiempo estimado: 3 a 4 semanas.

Errores comunes que debes evitar

- Guardar el Context de una Activity en un singleton o variable estática (memory leak clásico)
- No probar rotación de pantalla — el 90% de los bugs de ciclo de vida aparecen ahí
- Ignorar strings.xml y hardcodear texto (mala práctica que además rompe internacionalización)
- No entender que Fragment está en declive: hoy Google recomienda Activities + Compose Navigation para apps nuevas; aprende Fragment lo suficiente para mantenimiento de apps legacy, no lo profundices como base de tu arquitectura nueva

Proyecto obligatorio

App 'Diario de clase' (conectada a tu profesión): pantalla de lista de estudiantes, pantalla de detalle con navegación por Intent, persistencia de rotación de pantalla sin perder datos, soporte para modo oscuro y para español/inglés vía strings.xml, todo en XML/Views clásico (Compose viene en el siguiente nivel — es intencional que sientas la diferencia).

Desafío (después del proyecto)

Añade un permiso en runtime real (ej: acceso a contactos para importar una lista de estudiantes) manejando correctamente el caso de rechazo del permiso sin crashear.

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes dibujar de memoria el ciclo de vida de una Activity y explicar qué pasa en cada callback
- ☐ Tu app 'Diario de clase' sobrevive a una rotación de pantalla sin perder el estado
- ☐ Sabes explicar la diferencia entre Application Context y Activity Context con un ejemplo de cuándo cada uno causa un bug si se usa mal

Nivel 5 — Jetpack Compose (UI moderna, declarativa)

Objetivo

Compose es el estándar actual recomendado por Google para UI en Android — XML/Views sigue vivo en apps legacy pero todo proyecto nuevo profesional en 2026 se construye en Compose. Aprenderás a pensar en UI de forma declarativa (describes el 'qué', no el 'cómo'), un cambio de paradigma real, no solo una nueva sintaxis.

Conocimientos (todo lo que cubre esta etapa)

- Composable functions: @Composable, recomposición, por qué Compose recalcula solo lo necesario
- Estado: remember, mutableStateOf, state hoisting (el patrón central de Compose — el estado vive arriba, los eventos suben, el estado baja)
- Layouts fundamentales: Column, Row, Box, LazyColumn/LazyRow (equivalente moderno de RecyclerView)
- Modifiers: orden de ejecución (importa mucho, es una fuente común de bugs de UI), padding, size, clickable, etc.
- Material Design 3: Theming, ColorScheme, Typography, componentes (Button, TextField, Scaffold, TopAppBar)
- Navigation Compose: NavHost, NavController, paso de argumentos type-safe (con Kotlin Serialization en versiones recientes)
- Side-effects: LaunchedEffect, DisposableEffect, rememberCoroutineScope — cuándo y por qué usarlos
- Animaciones en Compose: animateFloatAsState, AnimatedVisibility, transiciones
- Slot APIs y composición de UI reutilizable
- Interoperabilidad Compose-Views (para trabajar en apps legacy que migran gradualmente)
- Responsive UI: WindowSizeClass para adaptar a tablets/foldables
- Accesibilidad en Compose: contentDescription, semantics

Recursos gratuitos

- Android Developers — 'Jetpack Compose Pathway' (codelabs oficiales completos y gratuitos, la mejor fuente única para este nivel)
- Compose Camp (Google, gratuito, con certificado de finalización)
- Philipp Lackner — serie completa de Compose en YouTube
- Compose sin Filtros / Compose Academy (comunidades hispanohablantes activas — buscar por estos nombres en YouTube/Discord)

Recursos de pago (solo los que realmente valen la pena)

- 'Jetpack Compose Masterclass' — Philipp Lackner (Udemy) — de los pocos cursos de pago que realmente recomiendo en todo el roadmap; profundiza más que los codelabs gratuitos en temas de estado y arquitectura
- 'Jetpack Compose: The Ultimate Guide' — CodingWithMitch — alternativa sólida

Orden exacto de estudio

- 1) Composables básicos y state hoisting (el concepto más importante de todo el nivel)
- 2) Layouts y modifiers
- 3) Material 3 theming completo
- 4) Listas (LazyColumn) con datos reales
- 5) Navigation Compose

- 6) Side-effects y animaciones

Tiempo estimado: 6 a 8 semanas.

Errores comunes que debes evitar

- No entender state hoisting y terminar con estado duplicado o desincronizado entre composables
- Ignorar el orden de los modifiers (padding antes de clickable da un resultado distinto que después)
- Poner lógica de negocio dentro de un Composable en vez de en el ViewModel (mezclar UI con lógica es el error arquitectónico más común en principiantes de Compose)
- No probar la recomposición con Layout Inspector — programar 'a ciegas' sin verificar recomposiciones innecesarias

Proyecto obligatorio

Reconstruye la app 'Diario de clase' del Nivel 4 completamente en Compose, agregando: tema claro/oscuro dinámico, lista de estudiantes con LazyColumn, formulario de registro con validación visual en tiempo real (reutilizando tu motor de validación del Nivel 2), navegación entre 3 pantallas con Navigation Compose, y al menos una animación (ej: al agregar un estudiante).

Desafío (después del proyecto)

Haz que la app sea responsive: debe verse bien tanto en teléfono como en tablet usando WindowSizeClass, mostrando una lista simple en teléfono y una vista maestro-detalle en tablet.

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes explicar 'state hoisting' con un dibujo simple sin ayuda
- ☐ Tu app no tiene ni una sola lógica de negocio escrita directamente dentro de un @Composable
- ☐ Usaste Layout Inspector al menos una vez para verificar recomposiciones
- ☐ Tu app se ve correctamente en modo oscuro y en tablet

Nivel 6 — Arquitectura profesional de aplicaciones

Objetivo

Esta es la etapa que separa a alguien que 'sabe programar Android' de alguien contratable como Mid/Senior. La arquitectura no es opcional en proyectos reales: sin ella, cualquier app de más de 5 pantallas se vuelve imposible de mantener. Aquí conectas todo lo aprendido (Kotlin, Compose, coroutines) en un sistema coherente.

Conocimientos (todo lo que cubre esta etapa)

- MVVM (Model-View-ViewModel): el patrón recomendado por Google para Android, cómo el ViewModel sobrevive a cambios de configuración
- MVI (Model-View-Intent): patrón unidireccional de estado, usado en apps grandes (Airbnb, Spotify) por su previsibilidad — cuándo MVI vale la complejidad extra frente a MVVM
- Clean Architecture aplicada a Android: capas (presentation, domain, data), regla de dependencia (las capas internas no conocen las externas)
- UseCases / Interactors: por qué aislar la lógica de negocio del ViewModel
- Repository pattern: abstracción de fuentes de datos (red, local, caché)
- Inyección de dependencias: por qué es indispensable para testing y desacoplamiento (no es 'complejidad innecesaria', es lo que permite testear sin mockear todo a mano)
- Hilt (la solución de DI recomendada por Google, construida sobre Dagger): módulos, componentes, scopes
- Koin como alternativa más simple (útil conocerla, muchas empresas medianas/freelance la usan por su curva de aprendizaje menor)
- Modularización: por qué separar el proyecto en múltiples módulos Gradle (feature modules, core modules) mejora tiempos de build y escalabilidad en equipo
- Unidirectional Data Flow (UDF): el estado fluye en una sola dirección, los eventos suben — principio central en toda arquitectura moderna de Android
- StateFlow/SharedFlow en el ViewModel como fuente única de verdad para la UI

Recursos gratuitos

- Android Developers — 'Guide to app architecture' (documentación oficial completa y actualizada, es LA referencia)
- Now in Android (github.com/android/nowinandroid) — app open source oficial de Google que demuestra arquitectura moderna real, código de referencia de altísima calidad para leer
- Philipp Lackner — serie de Clean Architecture en Android (YouTube, gratuita y completa)
- Hilt — Codelab oficial gratuito de Android Developers

Recursos de pago (solo los que realmente valen la pena)

- 'Android Clean Architecture' — Philipp Lackner (Udemy) — el curso de pago más recomendado de todo el roadmap; conecta arquitectura con proyecto real de principio a fin

Orden exacto de estudio

- 1) MVVM con ViewModel + StateFlow (domina esto primero, es la base de todo)
- 2) Repository pattern con una fuente de datos simple
- 3) UseCases y Clean Architecture completa

- 4) Inyección de dependencias con Hilt
- 5) MVI (como evolución, no como reemplazo — entiende cuándo elegir cada uno)
- 6) Modularización (solo cuando ya tengas un proyecto real que lo justifique — no modularices un proyecto de 3 pantallas)

Tiempo estimado: 8 a 10 semanas. No te apresures: leer sobre arquitectura es fácil, aplicarla correctamente sin ayuda toma repetición real en varios proyectos.

Errores comunes que debes evitar

- Copiar una estructura de carpetas de un tutorial sin entender la regla de dependencia detrás
- Meter lógica de negocio en el ViewModel en vez de en UseCases (funciona igual a corto plazo, pero no escala ni se testea igual de bien)
- Usar MVI en un proyecto pequeño donde MVVM simple habría bastado — sobre-ingeniería
- Modularizar prematuramente un proyecto pequeño solo 'porque así lo hacen las apps grandes'

Proyecto obligatorio

Reconstruye 'Diario de clase' con arquitectura Clean completa: capas domain/data/presentation, Repository con una fuente de datos remota simulada (ver Nivel 8), UseCases explícitos para cada acción del usuario, Hilt para toda la inyección de dependencias, y StateFlow como única fuente de verdad de cada pantalla.

Desafío (después del proyecto)

Migra el proyecto de MVVM a MVI puro: un solo estado inmutable por pantalla, un 'Intent' sellado que representa cada acción posible del usuario, y un reducer que transforma estado según el intent recibido.

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes dibujar de memoria las 3 capas de Clean Architecture y explicar la regla de dependencia
- ☐ Puedes justificar cuándo elegirías MVVM y cuándo MVI en una entrevista técnica
- ☐ Tu app usa Hilt sin errores de compilación y sin instanciar dependencias manualmente en ningún lugar
- ☐ Puedes leer el código de Now in Android y entender qué está pasando en cada capa

Nivel 7 — Persistencia de datos

Objetivo

Ninguna app real vive solo de memoria RAM. Necesitas saber elegir y usar la herramienta correcta de persistencia según el caso: preferencias simples, caché estructurado, o archivos.

Conocimientos (todo lo que cubre esta etapa)

- Room: entidades, DAOs, Database, relaciones (uno a muchos, muchos a muchos), migraciones de esquema
- DataStore (Preferences y Proto): reemplazo moderno de SharedPreferences, por qué es asíncrono y type-safe
- Diferencia clara: cuándo usar Room (datos estructurados, consultas complejas) vs DataStore (preferencias simples, flags, configuración)
- Flow con Room: observar cambios en la base de datos de forma reactiva
- Estrategia Offline-First: cachear datos localmente y sincronizar con red cuando hay conexión — patrón esperado en apps profesionales
- Testing de Room con bases de datos en memoria
- Encriptación de datos sensibles (introducción — se profundiza en Nivel 10 Seguridad)

Recursos gratuitos

- Android Developers — Codelab oficial de Room completo
- Android Developers — documentación de DataStore
- Philipp Lackner — serie de Room + Offline-First en YouTube

Recursos de pago (solo los que realmente valen la pena)

- No es necesario un curso de pago específico — la documentación oficial de Room es de las mejores de todo Android SDK

Orden exacto de estudio

- 1) DataStore Preferences (lo más simple primero)
- 2) Room básico: una tabla, CRUD completo
- 3) Room con relaciones
- 4) Flow reactivo con Room conectado a StateFlow del ViewModel
- 5) Patrón Offline-First combinando Room + Retrofit (necesitarás el Nivel 8 en paralelo aquí)

Tiempo estimado: 3 a 4 semanas.

Errores comunes que debes evitar

- Usar SharedPreferences en 2026 en vez de DataStore (obsoleto, se mantiene solo por compatibilidad legacy)
- Hacer queries de Room en el hilo principal (Room fuerza esto por diseño, pero muchos intentan evitarlo mal)
- No versionar las migraciones de base de datos desde el principio — causa crashes al actualizar la app en producción

Proyecto obligatorio

Convierte 'Diario de clase' para que persista completamente en Room: estudiantes, calificaciones, asistencia — con relaciones reales entre tablas (un estudiante tiene muchas calificaciones). Preferencias de la app (tema, idioma) en DataStore.

Desafío (después del proyecto)

Implementa Offline-First real: la app debe funcionar sin internet mostrando datos cacheados en Room, y sincronizar automáticamente cuando vuelve la conexión (usa el Nivel 8 para simular la fuente remota).

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes explicar cuándo usarías Room vs DataStore en una entrevista sin dudar
- ☐ Tu base de datos maneja al menos una relación uno-a-muchos correctamente
- ☐ Tu app no pierde datos si se cierra abruptamente mientras escribe en la base de datos

Nivel 8 — Networking

Objetivo

Casi ninguna app profesional vive aislada del backend. Necesitas dominar el consumo de APIs REST de forma robusta, manejando errores de red como ciudadano de primera clase del diseño, no como un 'catch' genérico al final.

Conocimientos (todo lo que cubre esta etapa)

- Retrofit: definición de interfaces de API, converters (Kotlin Serialization o Moshi — Gson está en declive, evítalo en proyectos nuevos)
- OkHttp: interceptors, logging, manejo de headers, timeouts
- Ktor Client como alternativa moderna 100% Kotlin (útil conocerlo, algunas empresas lo prefieren especialmente en proyectos multiplataforma)
- Manejo de estados de red: Loading/Success/Error modelados con sealed classes (conectando con lo aprendido en Nivel 2 y 6)
- Serialización JSON con kotlinx.serialization
- Manejo de errores de red: timeouts, sin conexión, códigos de error HTTP, reintentos (retry con backoff)
- Paginación: Paging 3 library para listas largas desde red
- Autenticación: tokens JWT, refresh tokens, interceptor de autenticación automática
- WebSockets básico (para casos de tiempo real, ej: chat)
- Certificate pinning (introducción — se profundiza en seguridad)

Recursos gratuitos

- Android Developers — Codelab oficial de Retrofit y networking
- Retrofit — documentación oficial de Square (square.github.io/retrofit)
- Philipp Lackner — serie de Retrofit + manejo de errores de red
- JSONPlaceholder / DummyJSON — APIs REST gratuitas para practicar sin necesitar backend propio

Recursos de pago (solo los que realmente valen la pena)

- No es necesario curso de pago dedicado — la combinación de documentación oficial + Philipp Lackner cubre esto completamente

Orden exacto de estudio

- 1) Retrofit básico con un GET simple contra una API pública gratuita
- 2) Manejo de estados (Loading/Success/Error) con sealed classes
- 3) POST/PUT/DELETE con serialización de objetos
- 4) Interceptors y manejo de autenticación
- 5) Paginación con Paging 3
- 6) Integración completa con Offline-First del Nivel 7

Tiempo estimado: 4 a 5 semanas.

Errores comunes que debes evitar

- No manejar el caso 'sin conexión a internet' explícitamente (la app simplemente 'no hace nada' en vez de informar al usuario)
- Exponer el modelo de red (DTO) directamente en la UI en vez de mapearlo a un modelo de dominio propio
- Ignorar timeouts — una API lenta puede colgar tu UI si no configuras límites
- Loggear tokens de autenticación en producción (riesgo de seguridad real)

Proyecto obligatorio

Convierte 'Diario de clase' en una app conectada a un backend real simulado: usa una API gratuita tipo JSONPlaceholder para simular estudiantes y calificaciones, implementa manejo de errores visual (banner de 'sin conexión', botón de reintentar), y sincroniza con Room del Nivel 7 (fuente remota + caché local).

Desafío (después del proyecto)

Implementa autenticación simulada con un interceptor que agregue un token a cada request y lo renueve automáticamente si expira (401), sin que el usuario tenga que volver a iniciar sesión manualmente en cada caso.

Checklist — cómo saber que dominaste esta etapa

- ☐ Tu app maneja correctamente los 4 estados de red: loading, éxito, error de servidor, sin conexión
- ☐ Nunca expones un modelo de red (DTO) directamente en un Composable — siempre pasa por un mapeo a modelo de dominio
- ☐ Puedes explicar qué es un interceptor y dar un ejemplo real de uso

Nivel 9 — Testing

Objetivo

El testing es lo que las empresas serias evalúan para diferenciar Mid de Senior, y es exactamente lo que la mayoría de freelancers/autodidactas se saltan — lo que te da una ventaja competitiva real si lo dominas.

Conocimientos (todo lo que cubre esta etapa)

- Pirámide de testing: unit tests (base, más cantidad), integration tests, UI tests (cima, menos cantidad pero críticos)
- JUnit 5: estructura de tests, assertions, setup/teardown
- MockK: mocking idiomático de Kotlin (mejor que Mockito para código Kotlin por cómo maneja coroutines y clases finales)
- Testing de coroutines: runTest, TestDispatcher
- Testing de ViewModels: verificar transiciones de estado sin depender de Android framework
- Testing de Repository con fake/mock de fuentes de datos
- Instrumentation tests: tests que corren en un dispositivo/emulador real
- Compose Testing: ComposeTestRule, finders (onNodeWithText, onNodeWithTag), assertions y acciones
- Test Doubles: diferencia entre mock, fake, stub, spy — cuándo usar cada uno
- TDD (Test-Driven Development): cuándo aplicarlo tiene sentido real en Android y cuándo no vale la pena forzarlo
- Cobertura de código: qué es un número útil vs una métrica vanidosa (100% cobertura no significa código sin bugs)

Recursos gratuitos

- Android Developers — 'Test apps on Android' (guía oficial completa)
- MockK — documentación oficial (mockk.io)
- Philipp Lackner — serie de testing en Android (unit + Compose testing)
- Now in Android (repo de Google) — tiene tests reales de referencia de altísima calidad para leer y aprender del código real

Recursos de pago (solo los que realmente valen la pena)

- 'Testing in Android' — cualquier curso específico de Udemy con buenas reseñas recientes si sientes que necesitas más estructura guiada; no es indispensable si sigues la documentación oficial con disciplina

Orden exacto de estudio

- 1) JUnit básico con funciones puras (Nivel 1-2 revisitado con enfoque en testing)
- 2) MockK: mockear una dependencia simple
- 3) Testing de ViewModel con coroutines (runTest)
- 4) Testing de Repository
- 5) Compose UI testing básico

Tiempo estimado: 4 semanas.

Errores comunes que debes evitar

- Escribir tests después de terminar toda la app (deja de ser sostenible, se vuelve una tarea gigante que se pospone indefinidamente)

- Testear detalles de implementación en vez de comportamiento (tests frágiles que se rompen con cualquier refactor menor)
- Perseguir 100% de cobertura como objetivo en sí mismo
- No testear casos de error, solo el 'happy path'

Proyecto obligatorio

Añade una suite de tests completa a 'Diario de clase': tests unitarios de cada UseCase, tests de ViewModel verificando las transiciones de estado (loading→success, loading→error), y al menos 3 Compose UI tests verificando que los elementos correctos aparecen en pantalla según el estado.

Desafío (después del proyecto)

Practica TDD real: para la siguiente funcionalidad nueva que agregues a la app, escribe el test ANTES que el código de producción, y deja que el test te guíe.

Checklist — cómo saber que dominaste esta etapa

- ☐ Tienes al menos 70% de cobertura real (no forzada) en la capa domain de tu proyecto
- ☐ Puedes explicar la diferencia entre mock, fake y stub con un ejemplo propio
- ☐ Tus tests de ViewModel no dependen de Android framework (corren como unit test puro, no instrumentado)

Nivel 10 — Seguridad

Objetivo

La seguridad no es un 'extra' — es lo que distingue una app profesional de un prototipo. Un freelancer o empleado que entrega apps inseguras genera responsabilidad legal y reputacional real para el cliente/empresa.

Conocimientos (todo lo que cubre esta etapa)

- OWASP Mobile Top 10: las vulnerabilidades más comunes en apps móviles y cómo prevenir cada una
- Almacenamiento seguro: EncryptedSharedPreferences/DataStore, Android Keystore System
- ProGuard/R8: ofuscación de código y reducción de tamaño de APK, configuración de reglas para evitar romper reflection (Room, Retrofit)
- Certificate pinning con OkHttp: prevenir ataques man-in-the-middle
- Manejo seguro de API keys: nunca hardcodeadas en el repositorio, uso de local.properties + BuildConfig, o mejor aún, backend proxy
- Validación de inputs del usuario (prevención básica de inyección, aunque en Android el vector principal suele ser distinto a web)
- Permisos: principio de mínimo privilegio, explicar al usuario por qué se necesita cada permiso
- Root detection y detección de emuladores (para apps financieras/sensibles)
- Biometric authentication API (huella/rostro) para apps que requieren autenticación fuerte

Recursos gratuitos

- OWASP Mobile Application Security (owasp.org/www-project-mobile-app-security) — recurso de referencia gratuito, mantenido activamente
- Android Developers — 'Security best practices' (guía oficial completa)
- Android Developers — documentación oficial de Keystore y BiometricPrompt

Recursos de pago (solo los que realmente valen la pena)

- No hay un curso de pago que recomiende específicamente aquí — esta es un área donde documentación oficial + OWASP es superior a la mayoría de cursos comerciales

Orden exacto de estudio

- 1) OWASP Mobile Top 10 (lectura y comprensión, no código todavía)
- 2) Almacenamiento seguro con EncryptedSharedPreferences/Keystore
- 3) Configuración de ProGuard/R8 en un proyecto real
- 4) Certificate pinning
- 5) Biometric authentication

Tiempo estimado: 3 semanas.

Errores comunes que debes evitar

- Subir API keys al repositorio de GitHub (error extremadamente común y grave — usa .gitignore desde el día 1 de cada proyecto)

- Activar minifyEnabled sin probar exhaustivamente (R8 puede romper reflection de Room/Retrofit si las reglas de ProGuard están mal configuradas)
- Asumir que 'nadie va a hacer reverse engineering de mi app pequeña' — la ofuscación es barata de implementar, no hay razón para omitirla

Proyecto obligatorio

Audita y asegura 'Diario de clase': mueve cualquier credencial a local.properties, implementa EncryptedSharedPreferences para datos sensibles (si guardas algo tipo notas privadas del docente sobre estudiantes), configura R8 correctamente y verifica que el release build funcione idéntico al debug, y añade autenticación biométrica opcional para abrir la app.

Desafío (después del proyecto)

Implementa certificate pinning contra la API que usaste en el Nivel 8 y documenta qué pasaría si el certificado del servidor cambia (para que entiendas el trade-off de mantenimiento que implica).

Checklist — cómo saber que dominaste esta etapa

- ☐ Ninguna API key o secreto está en tu repositorio de GitHub (verifica el historial completo, no solo el commit actual)
- ☐ Tu build de release con R8 activado funciona exactamente igual que tu build de debug
- ☐ Puedes nombrar y explicar al menos 5 de los 10 riesgos de OWASP Mobile sin consultar la lista

Nivel 11 — Optimización y rendimiento

Objetivo

Una app que funciona no es lo mismo que una app que funciona bien. El rendimiento es medible, y las empresas usan herramientas de profiling reales en revisiones de código — necesitas saber usarlas, no solo 'programar con cuidado'.

Conocimientos (todo lo que cubre esta etapa)

- Android Studio Profiler: CPU, memoria, red, energía — cómo leer cada uno
- Memory leaks: causas comunes (contexto estático, listeners no removidos, coroutines sin cancelar) y cómo detectarlos con LeakCanary
- Recomposición en Compose: cómo minimizarla (stability, keys en listas, derivedStateOf)
- Baseline Profiles: cómo mejorar el tiempo de arranque de la app (startup time) — técnica moderna recomendada por Google
- R8 y reducción de tamaño de APK/AAB (App Bundle)
- Optimización de imágenes: formatos modernos (WebP/AVIF), carga eficiente con Coil
- Gestión eficiente de listas grandes: LazyColumn con keys correctas, paginación
- Macrobenchmark y Microbenchmark libraries para medir rendimiento de forma objetiva, no intuitiva
- ANR (Application Not Responding) — profundización: cómo detectarlos con Play Console antes de que los usuarios se quejen

Recursos gratuitos

- Android Developers — 'Performance' (sección completa de la documentación oficial)
- Android Developers — Codelab de Baseline Profiles
- LeakCanary — documentación oficial de Square (herramienta gratuita indispensable)
- Google I/O talks sobre rendimiento en Android (YouTube, canal oficial de Android Developers)

Recursos de pago (solo los que realmente valen la pena)

- No es necesario curso de pago — esta área depende más de práctica con las herramientas que de contenido teórico adicional

Orden exacto de estudio

- 1) Instalar y usar LeakCanary en un proyecto real, encontrar y arreglar un leak intencional
- 2) Android Studio Profiler: CPU y memoria en tu app 'Diario de clase'
- 3) Optimización de recomposición en Compose
- 4) Baseline Profiles
- 5) Optimización de imágenes con Coil

Tiempo estimado: 3 a 4 semanas.

Errores comunes que debes evitar

- Optimizar sin medir primero ('optimización prematura') — siempre perfila antes de cambiar código por sospecha
- Ignorar memory leaks pequeños porque 'la app no crashea' — se acumulan y crashean en sesiones largas
- No probar en un dispositivo de gama baja — tu emulador potente esconde problemas de rendimiento reales

Proyecto obligatorio

Perfila completamente 'Diario de clase': encuentra y corrige al menos un memory leak real usando LeakCanary, reduce recomposiciones innecesarias verificándolo con Layout Inspector, implementa Baseline Profile y mide la mejora de tiempo de arranque antes/después.

Desafío (después del proyecto)

Prueba tu app en un emulador configurado para simular un dispositivo de gama baja (poca RAM, CPU limitada) y documenta qué problemas de rendimiento aparecen que no veías en tu configuración normal.

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes leer un reporte de LeakCanary y explicar la causa raíz de un leak
- ☐ Puedes explicar qué es un Baseline Profile y por qué mejora el startup time
- ☐ Tu app no muestra frames perdidos (jank) visibles al hacer scroll en una lista de 100+ elementos

Nivel 12 — Publicación y distribución

Objetivo

Un proyecto que nunca llega a producción no cuenta como experiencia real ante un cliente o empleador. Aquí aprendes todo el proceso de llevar una app desde tu máquina hasta las manos de un usuario real.

Conocimientos (todo lo que cubre esta etapa)

- Google Play Console: creación de cuenta de desarrollador, ficha de la app, políticas de contenido
- Firma de aplicaciones: keystore, Play App Signing
- Android App Bundle (AAB) vs APK: por qué AAB es el formato requerido actualmente
- Versionado semántico: versionCode vs versionName, estrategia de releases
- Testing tracks: internal testing, closed testing, open testing, producción — flujo profesional de release gradual
- Play Console: pre-launch reports, vitals (crashes, ANRs), políticas de revisión de Google
- In-app purchases y suscripciones con Play Billing Library
- AdMob: integración de anuncios (relevante si el modelo de negocio de un cliente lo requiere)
- Firebase: Crashlytics (monitoreo de crashes en producción), Analytics (comportamiento de usuario), Remote Config (cambiar comportamiento sin publicar nueva versión), Authentication
- Push notifications con Firebase Cloud Messaging
- CI/CD: GitHub Actions para automatizar build/test, Fastlane para automatizar publicación

Recursos gratuitos

- Android Developers — 'Publish your app' (guía oficial completa paso a paso)
- Firebase — documentación oficial (todas las herramientas mencionadas tienen tier gratuito generoso)
- GitHub Actions — documentación oficial + 'Awesome Actions' (repositorio con ejemplos gratuitos para Android)
- Fastlane — documentación oficial (fastlane.tools)

Recursos de pago (solo los que realmente valen la pena)

- Cuenta de desarrollador de Google Play — costo único de registro (no es opcional, es el costo real de entrada al negocio, considéralo una inversión de tu freelancing, no un curso)

Orden exacto de estudio

- 1) Configurar Firebase completo en 'Diario de clase' (Crashlytics + Analytics)
- 2) Configurar CI básico con GitHub Actions (build + tests automáticos en cada push)
- 3) Generar tu primer AAB firmado
- 4) Publicar en internal testing en Play Console
- 5) Configurar Fastlane para automatizar el proceso de release

Tiempo estimado: 2 a 3 semanas.

Errores comunes que debes evitar

- Perder el keystore original (sin backup) — si pasa, no puedes actualizar tu app nunca más con Play App Signing mal configurado, es uno de los errores más costosos posibles

- No leer las políticas de Google Play antes de publicar — rechazos por incumplimiento de políticas son comunes y frustrantes si no las conoces
- No configurar Crashlytics desde el día 1 de producción — te enteras de crashes por reseñas negativas en vez de por el dashboard

Proyecto obligatorio

Publica 'Diario de clase' en Google Play Console en modo Internal Testing (no necesitas llegar a producción pública si el contenido es solo demostrativo), con Firebase Crashlytics y Analytics configurados, y un pipeline de GitHub Actions que corra tus tests automáticamente en cada push.

Desafío (después del proyecto)

Automatiza el proceso completo de release con Fastlane: un solo comando debe generar el AAB, subirlo a Play Console, e incrementar automáticamente el versionCode.

Checklist — cómo saber que dominaste esta etapa

- ☐ Tienes un backup seguro (fuera de tu computadora) de tu keystore de firma
- ☐ Tu app está publicada al menos en Internal Testing con Crashlytics reportando datos reales
- ☐ Tu pipeline de CI corre tests automáticamente en cada push a GitHub sin intervención manual

Nivel 13 — Freelancing: de desarrollador a negocio

Objetivo

Saber programar no es lo mismo que saber vender tu trabajo de programar. Esta etapa es tan importante como cualquier etapa técnica — es donde conviertes la habilidad en ingresos reales.

Conocimientos (todo lo que cubre esta etapa)

- Plataformas: Workana y Upwork (tu enfoque ya definido) — diferencias reales entre ambas, tipo de cliente típico en cada una
- Perfil optimizado: portafolio visible primero, no certificados — un reclutador/cliente decide en los primeros 10 segundos
- Cómo redactar propuestas (cover letters) que se leen completas: personalización real al proyecto, no plantillas genéricas copiadas
- Cotización de proyectos: por hora vs por proyecto cerrado — cuándo usar cada modelo y cómo protegerte de 'scope creep'
- Estimación de tiempos: por qué los desarrolladores subestiman sistemáticamente (ley de Hofstadter) y cómo compensarlo con buffers realistas
- Contratos básicos: qué debe incluir un contrato freelance (alcance, entregables, forma de pago, propiedad intelectual, cláusula de revisiones incluidas)
- Negociación: cómo defender tu tarifa sin perder el cliente, cómo identificar cuándo un cliente no vale la pena
- Red flags de clientes problemáticos: pedidos de trabajo 'de prueba' gratis extenso, presión de precio agresiva desde el primer mensaje, falta de claridad en el alcance
- Cómo cobrar internacionalmente desde Colombia: Payoneer, Wise, particularidades fiscales a considerar (consulta con un contador local para tu caso específico — esto no es asesoría fiscal)
- Construcción de reputación: primeras reseñas (a veces vale la pena un primer proyecto con margen bajo para conseguir la primera reseña sólida), consistencia en comunicación
- Tu ventaja competitiva real: eres docente. Hay una demanda concreta de apps educativas (gestión escolar, apps para padres, plataformas de contenido pedagógico) donde entiendes el dominio del problema mejor que un desarrollador genérico — esto es un diferenciador de nicho real, no un relleno de perfil

Recursos gratuitos

- Guías oficiales de ayuda de Workana y Upwork sobre cómo optimizar perfil y propuestas
- Indie Hackers (indiehackers.com) — comunidad gratuita con historias reales de freelancers/indie devs, útil para calibrar expectativas realistas
- r/freelance y r/androiddev (Reddit) — para entender el mercado real desde adentro, con matices y sesgos propios de foros que debes filtrar con criterio

Recursos de pago (solo los que realmente valen la pena)

- Ningún curso es indispensable aquí — la mejor forma de aprender freelancing es empezar con 1-2 proyectos reales de bajo riesgo y ajustar. Si quieres estructura, 'The Freelancer's Bible' (libro) da un marco general útil aunque no específico de Android

Orden exacto de estudio

- 1) Construye tu portafolio (ver sección de Portafolio más abajo) ANTES de crear perfiles en las plataformas

- 2) Crea perfiles en Workana y Upwork con portafolio visible
- 3) Practica escribir 5 propuestas para proyectos reales publicados (aunque no las envíes todavía) para calibrar tu criterio
- 4) Define tu tarifa inicial realista para el mercado colombiano/LATAM freelance en Android
- 5) Envía tus primeras propuestas reales, prioriza 1-2 proyectos pequeños para construir reputación antes de ir por proyectos grandes

Tiempo estimado: Proceso continuo desde que tengas 3-4 apps sólidas en tu portafolio (aproximadamente a partir del mes 10-12 del roadmap completo, no antes).

Errores comunes que debes evitar

- Empezar a buscar clientes antes de tener un portafolio real — quema oportunidades con una primera impresión débil
- Cotizar demasiado bajo 'para conseguir el primer cliente' de forma sistemática — devalúa tu trabajo y atrae al peor tipo de cliente
- Aceptar proyectos sin alcance claro por escrito ('lo vemos sobre la marcha') — es la causa número uno de conflictos con clientes freelance
- No pedir anticipo/hito de pago en proyectos largos con clientes nuevos sin historial

Proyecto obligatorio

No es un proyecto de código: es tu perfil completo en Workana y/o Upwork, con portafolio enlazado (GitHub + al menos 1 app en Play Store), descripción de servicios clara, y 3 propuestas reales enviadas a proyectos publicados.

Desafío (después del proyecto)

Cierra tu primer proyecto freelance real, aunque sea pequeño y de margen bajo. La primera reseña vale más que la primera ganancia grande.

Checklist — cómo saber que dominaste esta etapa

- ☐ Tienes un perfil profesional completo en al menos una plataforma freelance
- ☐ Puedes explicar en 30 segundos tu propuesta de valor como 'desarrollador Android con background docente' a un cliente potencial
- ☐ Tienes al menos un borrador de contrato/acuerdo de alcance que usarías con un cliente nuevo

Nivel 14 — Nivel Senior: lo que realmente te falta después de dominar lo técnico

Objetivo

La diferencia entre Mid y Senior rara vez es 'conocer más APIs'. Es juicio: saber cuándo NO usar una tecnología, comunicar trade-offs a personas no técnicas, y sostener la calidad del código bajo presión de tiempo real.

Conocimientos (todo lo que cubre esta etapa)

- Trade-off thinking: todo Senior evalúa costo/beneficio de cada decisión técnica en contexto de negocio, no en abstracto ('¿MVI aquí vale la complejidad para este cliente con 2 pantallas?')
- Comunicación técnica-no técnica: explicar por qué algo toma más tiempo del esperado a un cliente o product manager sin tecnicismos
- Code review real: dar feedback que mejora el código sin ser destructivo, y recibir feedback sin actitud defensiva
- Deuda técnica: identificarla, comunicarla, priorizarla frente a features nuevas — un Senior sabe cuándo pagarla y cuándo conscientemente posponerla
- Mentoría: la capacidad de explicar conceptos a otros (como estás documentando tu propio aprendizaje) es una habilidad Senior en sí misma, practícala ayudando a otros en comunidades
- Toma de decisiones bajo ambigüedad: requisitos incompletos son la norma en proyectos reales, no la excepción — un Senior pregunta lo correcto y avanza con supuestos documentados en vez de bloquearse
- Arquitectura pragmática: saber simplificar cuando el contexto lo permite en vez de aplicar 'la arquitectura perfecta' en cada proyecto sin importar su tamaño
- Entrevistas técnicas: system design aplicado a mobile (¿cómo diseñarías la sincronización offline de una app de notas colaborativa?), preguntas de comportamiento (STAR method), y cómo hablar de proyectos propios con profundidad real

Recursos gratuitos

- 'Staff Engineer' de Will Larson — hay resúmenes y capítulos gratuitos ampliamente compartidos; el libro completo es de pago pero vale cada peso si puedes acceder a él
- Charlas de Google I/O y Droidcon (YouTube, gratuitas) sobre arquitectura y decisiones de escala real en empresas grandes
- Contribuir a proyectos open source de Android (una de las formas más rápidas y gratuitas de exponerte a code review real de desarrolladores Senior)

Recursos de pago (solo los que realmente valen la pena)

- 'Staff Engineer' de Will Larson (libro completo) — la referencia más citada en la industria sobre qué distingue a ingenieros senior/staff más allá de la técnica pura

Orden exacto de estudio

- 1) Practica explicar cada decisión técnica de tus proyectos anteriores en términos de trade-offs, no de 'la forma correcta de hacerlo'
- 2) Contribuye a al menos un proyecto open source de Android (empieza con issues etiquetados 'good first issue')
- 3) Practica entrevistas técnicas simuladas conmigo (te las voy a plantear cuando llegues aquí)

- 4) Ayuda a alguien más en una comunidad (Discord/Reddit de Android hispanohablante) — enseñar consolida lo que sabes

Tiempo estimado: Continuo — este nivel no 'termina', es una forma de operar que sigues refinando con cada proyecto real.

Errores comunes que debes evitar

- Confundir 'Senior' con 'conoce más librerías' — Senior es sobre todo juicio y comunicación, no volumen de conocimiento
- Evitar dar o recibir feedback por incomodidad — es la habilidad que menos se practica y más se necesita
- No documentar decisiones arquitectónicas (Architecture Decision Records) — un Senior deja rastro de por qué se decidió algo, no solo el qué

Proyecto obligatorio

Escribe 3 Architecture Decision Records (ADRs) reales sobre decisiones que tomaste en 'Diario de clase' a lo largo del roadmap (ej: por qué MVVM y no MVI, por qué Room y no solo DataStore), explicando el contexto, las alternativas consideradas, y el trade-off elegido.

Desafío (después del proyecto)

Realiza una entrevista técnica simulada completa conmigo: system design de una app mobile con requisitos ambiguos que yo te voy a dar, y defiende tus decisiones en tiempo real como en una entrevista real.

Checklist — cómo saber que dominaste esta etapa

- ☐ Puedes defender cualquier decisión de arquitectura de tu proyecto principal ante alguien que la cuestione, sin ponerte defensivo
- ☐ Has hecho al menos una contribución real (por pequeña que sea) a un proyecto open source
- ☐ Puedes resolver un problema de system design mobile básico en una sesión simulada sin bloquearte por la ambigüedad de los requisitos

Git profesional (más allá de commit / push / pull)

Usar Git a nivel profesional es una habilidad separada de saber programar, y es una de las cosas que más rápido delata a un desarrollador junior en una entrevista técnica o en una revisión de código real.

Contenido

- Git Flow: ramas main/develop/feature/release/hotfix — el modelo clásico usado en proyectos con releases planificados
- Trunk Based Development: alternativa moderna preferida por equipos con CI/CD maduro (ramas de vida muy corta, integración continua a main) — entiende cuándo cada modelo tiene sentido, no hay un 'correcto' universal
- Rebase vs Merge: cuándo usar cada uno, por qué rebase mantiene un historial lineal más limpio pero reescribe historia (peligroso en ramas compartidas)
- Squash: combinar múltiples commits en uno antes de mergear, para un historial legible
- Cherry-pick: aplicar un commit específico de una rama a otra sin mergear todo
- Git Bisect: búsqueda binaria automatizada para encontrar el commit exacto que introdujo un bug
- Git Hooks: pre-commit (ej: correr lint automáticamente antes de cada commit), pre-push
- Submodules: incluir un repositorio dentro de otro (poco común en Android puro, pero útil conocerlo para proyectos multi-repo)
- Resolución de conflictos: leer un conflicto de merge sin pánico, entender <<<<<<< / ======= / >>>>>>>
- Buenas prácticas de commits: mensajes descriptivos (Conventional Commits: feat/fix/refactor/docs), commits atómicos (un cambio lógico por commit)
- Pull Requests: cómo escribir una descripción de PR que facilite la revisión, cómo responder a comentarios de revisión sin actitud defensiva

Recursos

Pro Git (libro oficial, 100% gratuito, git-scm.com/book) es el recurso más completo que existe. GitHub Skills (skills.github.com) para practicar de forma interactiva y gratuita. No necesitas ningún curso de pago para Git.

Cómo se integra al roadmap

No lo aprendas todo de golpe: Git Flow básico y buenas prácticas de commits desde el Nivel 0. Rebase, squash y bisect cuando ya tengas proyectos reales con historial (a partir del Nivel 4-5). Hooks cuando integres CI/CD en el Nivel 12.

Matemáticas: qué necesitas realmente (y qué no)

La buena noticia: Android no requiere matemáticas avanzadas para el 95% del desarrollo de apps de negocio (que es la mayoría del mercado freelance). No necesitas cálculo ni álgebra lineal salvo que te especialices en un nicho específico.

Contenido

- Aritmética y porcentajes: básico, para cálculos de negocio (precios, descuentos, estadísticas simples en apps)
- Lógica booleana: fundamental — es literalmente la base de todo if/when/condicional
- Geometría básica de coordenadas (x, y, ancho, alto): necesaria para layouts, animaciones, Canvas en Compose
- Trigonometría básica (seno/coseno): SOLO si trabajas con animaciones circulares, gráficos personalizados con Canvas, o Google Maps con cálculos de distancia/rumbo — no es prioritario para el 80% de apps
- Notación Big O: técnicamente es análisis de algoritmos, no 'matemáticas' en el sentido tradicional, pero es la matemática más relevante que usarás — ver sección de Algoritmos
- Probabilidad y estadística básica: útil si tu app tiene analytics, A/B testing, o features de gamificación con probabilidades

Recursos

Khan Academy (gratuito, completo) para cualquier repaso de aritmética/geometría/trigonometría básica si sientes vacíos. No necesitas ir más allá salvo que un proyecto específico lo requiera (ej: un cliente pide una app con visualización de datos científicos compleja).

Cómo se integra al roadmap

No dediques tiempo dedicado a matemáticas como bloque separado del roadmap — apréndelas 'justo a tiempo' cuando un proyecto específico las requiera (ej: trigonometría cuando llegues a animaciones avanzadas en Compose).

Algoritmos y estructuras de datos (plan paralelo)

Esto SÍ importa, pero por una razón distinta a la que crees: no lo vas a usar todos los días construyendo apps, pero es lo que evalúan en entrevistas técnicas de empresas medianas/grandes, y desarrolla tu capacidad de razonar sobre eficiencia de código real.

Contenido

- Estructuras de datos fundamentales: Array, LinkedList, Stack, Queue, HashMap, Tree (especialmente árboles binarios), Graph — cuándo usar cada una y por qué (ya usas varias de estas indirectamente en Kotlin Collections)
- Complejidad algorítmica (Big O): O(1), O(log n), O(n), O(n log n), O(n^2) — saber estimar la complejidad de tu propio código, no solo resolver ejercicios de LeetCode
- Algoritmos de búsqueda: búsqueda lineal vs binaria
- Algoritmos de ordenamiento: entender al menos uno de cada familia (burbuja para entender el concepto, quicksort/mergesort para entender los eficientes — no necesitas implementarlos de memoria en el trabajo real, Kotlin ya los trae optimizados, pero debes entender por qué sort() de Kotlin es rápido)
- Recursividad: base de árboles, backtracking, y muchos problemas de entrevista
- Two pointers, sliding window: patrones comunes en problemas de arrays/strings

Recursos

NeetCode.io (gratuito en su mayoría, la mejor guía estructurada de patrones de entrevista actual) · LeetCode (plan gratuito suficiente, no necesitas Premium al inicio) · 'Grokking Algorithms' de Aditya Bhargava (libro, el más amigable visualmente para autodidactas, versión de pago pero de bajo costo y alto valor).

Cómo se integra al roadmap

Plan paralelo real: dedica 2-3 problemas por semana desde el Nivel 2 en adelante (cuando ya domines Kotlin básico), NO en bloques maratónicos. La meta no es 'terminar LeetCode', es desarrollar intuición de eficiencia. Intensifica esto (4-5 problemas/semana) 6-8 semanas antes de empezar a postularte a empleos formales (no antes de buscar clientes freelance — ahí casi nunca se evalúa esto).

Inglés técnico: el nivel real que necesitas

No necesitas inglés fluido conversacional para programar — necesitas inglés de LECTURA técnica sólido (documentación, mensajes de error, código) desde el día 1, e inglés de conversación decente si apuntas a clientes internacionales o empleo remoto en empresa extranjera.

Contenido

- Nivel de lectura técnica: debes poder leer documentación oficial de Android/Kotlin sin traducir mentalmente palabra por palabra — esto se logra exponiéndote constantemente a documentación en inglés desde el Nivel 0, no pausando el roadmap para 'primero aprender inglés'
- Vocabulario técnico específico: términos que se repiten constantemente (deprecated, thread-safe, race condition, memory leak, boilerplate, edge case) — se aprenden por repetición natural leyendo documentación y código
- Inglés escrito para propuestas freelance en Upwork: nivel B1-B2 es suficiente para escribir propuestas claras y profesionales — usa herramientas de asistencia (Grammarly, o pide revisión a una IA) sin depender de ellas ciegamente
- Inglés oral: solo crítico si apuntas a entrevistas con empresas extranjeras o llamadas con clientes internacionales — en Workana/Upwork la mayoría de comunicación es escrita

Recursos

No pauses el roadmap para 'estudiar inglés primero' — la inmersión técnica (documentación 100% en inglés desde el Nivel 0) es más efectiva que un curso genérico separado. Si tu nivel de lectura técnica es realmente bajo, un mes de repaso de gramática básica en Duolingo/Cambly antes de empezar puede ayudar, pero no lo extiendas más de eso.

Cómo se integra al roadmap

Toda la documentación oficial que se recomienda en este roadmap está en inglés intencionalmente — es tu entrenamiento de inglés técnico integrado en el aprendizaje, no un bloque aparte.

Cómo usar IA (ChatGPT, Gemini, Claude) sin volverte dependiente

La IA es una herramienta que acelera brutalmente el aprendizaje mal usada te deja sin criterio propio. La diferencia está en el momento y la forma en que la usas.

Contenido

- Úsala para: explicar un concepto que no entendiste de la documentación con otras palabras (pero verifica contra la documentación oficial después)
- Úsala para: revisar tu código ya escrito por ti y sugerir mejoras (nunca generar código completo de un proyecto de aprendizaje desde cero)
- Úsala para: generar ejercicios de práctica adicionales sobre un tema específico donde sientes debilidad
- Úsala para: simular entrevistas técnicas y recibir feedback
- Úsala para: debug — pegar un error y entender qué significa (pero escribe tú mismo la corrección, no copies la solución generada sin escribirla a mano)
- NO la uses para: generar proyectos completos de tu portafolio — un reclutador puede notar código genérico de IA, y aunque no lo note, tú no aprendiste nada y no podrás defenderlo en una entrevista técnica
- NO la uses para: evitar leer documentación oficial — la documentación te da precisión y contexto que un resumen de IA puede simplificar de más o directamente equivocar
- Regla práctica: si no puedes explicar sin ayuda una línea de código que 'generaste' con IA, no la uses en tu proyecto todavía

Recursos

No se requieren recursos de pago para esto — el uso correcto es una disciplina personal, no una herramienta a comprar.

Cómo se integra al roadmap

A partir del Nivel 6 en adelante (cuando ya tienes bases sólidas) la IA se vuelve más útil como acelerador de arquitectura y debugging. En los Niveles 0-3, úsala con moderación extrema — es cuando más fácil es generar dependencia que reemplaza el aprendizaje real de fundamentos.

Rutina semanal sostenible (compatible con tu jornada docente)

Tu jornada docente son ~6 horas diarias, más familia y descanso. Un plan de estudio realista para ti NO es de 4 horas diarias — es de 1.5 a 2.5 horas diarias entre semana con más tiempo el fin de semana, sostenido por muchos meses, no semanas intensas seguidas de abandono.

Distribución semanal

- Lunes a viernes (días de clase): 1.5 a 2 horas — bloque único después del trabajo y antes de estar agotado, idealmente la misma hora todos los días para construir el hábito. Usa Pomodoro (25 min enfoque / 5 min descanso, 3-4 ciclos) para sostener concentración después de una jornada docente física y mentalmente demandante.
- Sábado: 3 a 4 horas — tu bloque de Deep Work (trabajo profundo sin interrupciones) para lo que requiere más concentración: arquitectura, debugging complejo, avanzar en el proyecto de la etapa actual.
- Domingo: 1.5 a 2 horas — repaso espaciado (revisar notas/conceptos de la semana, NO contenido nuevo) + planificación de la semana siguiente. El domingo es también tu día de descanso real — no lo satures.
- Descanso: al menos 1 día completo sin código a la semana si sientes fatiga acumulada — la sostenibilidad de 12-24 meses depende más de evitar el burnout que de maximizar horas semanales.

Técnicas de estudio a aplicar

- Repetición espaciada: revisa conceptos del Nivel N-1 brevemente cuando empieces el Nivel N (10-15 min), y conceptos de 2-3 niveles atrás una vez al mes — esto es lo que evita que 'olvides Kotlin mientras aprendes Compose'
- Pomodoro entre semana, Deep Work el sábado: no uses Pomodoro para arquitectura compleja (interrumpe el flow), resérvalo para tareas más mecánicas (ejercicios, Koans, testing)
- Regla del 80/20 aplicada: cuando un recurso (curso/video) se vuelve redundante con lo que ya sabes, sáltalo — no completes contenido por completitud, complétalo por necesidad real de aprendizaje
- Un solo proyecto activo por nivel: no empieces el proyecto del siguiente nivel hasta cerrar el checklist del nivel actual — el cambio de contexto constante es lo que generalmente lleva al abandono (como pasó con Flutter)

Portafolio progresivo: 25 aplicaciones

No necesitas 25 apps complejas — necesitas una progresión visible de complejidad creciente que cuente una historia clara a un reclutador o cliente: 'este desarrollador creció de fundamentos a producción real'. Muchas de estas ya están cubiertas como proyectos de cada nivel; aquí está el mapa completo.

Fase 1 — Fundamentos (apps 1-8, Niveles 0-4)

CLI de gestor de gastos · CLI de biblioteca escolar · Calculadora con historial · Conversor de unidades · Validador de formularios (librería) · Lista de tareas simple en XML/Views · App de notas con ciclo de vida robusto · Diario de clase v1 (XML/Views)

Fase 2 — Compose y arquitectura (apps 9-15, Niveles 5-7)

Diario de clase v2 (Compose) · Clon simplificado de una app de hábitos (con animaciones) · App de recetas con LazyColumn y detalle (patrón maestro-detalle) · Diario de clase v3 (Clean Architecture + Hilt) · App de notas con Room + relaciones · Gestor de asistencia escolar offline-first · App de calificaciones con DataStore para preferencias

Fase 3 — Producto real (apps 16-21, Niveles 8-11)

Diario de clase v4 (conectado a API real, con auth) · Clon de app de clima consumiendo API pública real · App de noticias con paginación (Paging 3) · App de chat básico (WebSockets) · Diario de clase v5 (testeado, seguro, optimizado — versión 'production-ready') · Una app 100% original de tu autoría resolviendo un problema real del contexto educativo colombiano (esta es la pieza central de tu portafolio de nicho)

Fase 4 — Publicadas y de nicho freelance (apps 22-25, Niveles 12-14)

Publica en Play Store al menos 2 de las apps anteriores (idealmente la de nicho educativo y una más) · Una app hecha a la medida de un caso de uso hipotético de cliente freelance (ej: sistema de reservas para un negocio local de Santa Marta, con backend real) · Una contribución open source documentada como parte del portafolio (aporta a Now in Android o un proyecto similar) · Una app case study completa: documento explicando el problema, arquitectura, decisiones (ADRs) y resultado — esto es lo que un cliente/reclutador Senior realmente lee con atención, más que el código mismo

Nota estratégica

La app de nicho educativo (Fase 3) es tu diferenciador competitivo más importante: como docente, conoces problemas reales que un desarrollador genérico no ve — gestión de calificaciones, comunicación con padres, seguimiento de asistencia, planeación de clases. Esa comprensión del dominio vale tanto como el código.

Empleabilidad: qué evalúan realmente las empresas

No es lo que dicen los cursos ('aprende 20 tecnologías y consigue trabajo'). Es esto, en este orden de importancia real:

- 1) Código que pueden entender otros: variables con nombres claros, funciones con una responsabilidad, PRs pequeños y revisables — más que 'conocer muchas librerías'
- 2) Capacidad de explicar decisiones: en una entrevista técnica te preguntan MENOS 'qué hace tal función de Kotlin' y MÁS '¿por qué elegiste esta arquitectura para este problema?'
- 3) Testing real, no solo mencionado en el CV: un repositorio con tests reales pesa más que un certificado
- 4) Git limpio: historial de commits legible es una señal fuerte de profesionalismo que se revisa en cualquier entrevista técnica seria que incluya revisión de código
- 5) Un proyecto completo y pulido pesa más que diez proyectos a medias: profundidad sobre volumen, siempre
- 6) Comunicación clara en español Y en inglés técnico escrito, si apuntas a empresas remotas internacionales
- 7) Comportamiento en la entrevista de código en vivo: cómo piensas en voz alta y manejas no saber algo (pedir una pista, razonar el problema) importa más que llegar a la solución perfecta al primer intento

Nota sobre certificaciones

Las certificaciones (Udemy, Coursera) prácticamente no se verifican ni pesan en decisiones de contratación real en la industria — son útiles para tu propio aprendizaje estructurado, no como credencial. Esto confirma lo que ya se identificó en tu estrategia de datos: portafolio en GitHub/Play Store por encima de certificados.

Plan de acción cronológico (12 a 24 meses)

Hitos mensuales con objetivo, entregable concreto y métrica de progreso verificable. Este plan asume tu ritmo sostenible definido en la sección de Productividad — ajústalo si tu contexto cambia, pero no lo abandones sin reemplazarlo por otro plan igual de concreto.

Mes 1 — Nivel 0 (preparación) + inicio Nivel 1 (fundamentos de programación)

Hito: Entorno instalado, primer repositorio en GitHub, horario semanal definido y respetado 4 semanas seguidas.

Métrica de progreso: Kotlin Koans sección 'Introduction' completa.

Mes 2 — Cierre Nivel 1 + inicio Nivel 2 (Kotlin núcleo: null safety, data/sealed classes)

Hito: Proyecto CLI de gestor de gastos terminado y funcionando.

Métrica de progreso: Puedes explicar null safety y sealed classes sin apoyo.

Mes 3 — Nivel 2 continuación (collections, generics, extension functions)

Hito: Ninguno crítico — mes de consolidación densa.

Métrica de progreso: Resuelves ejercicios de programación funcional (map/filter/reduce) sin dudar.

Mes 4 — Nivel 2 cierre (coroutines, Flow básico) + Nivel 3 (POO y SOLID)

Hito: Motor de validación de formularios terminado. Sistema de biblioteca escolar con SOLID aplicado.

Métrica de progreso: Checklist completo de Nivel 2 y Nivel 3.

Mes 5 — Nivel 4 (Android SDK: ciclo de vida, manifest, recursos)

Hito: App 'Diario de clase' v1 (XML/Views) funcionando con navegación e internacionalización.

Métrica de progreso: Sobrevive rotación de pantalla sin perder estado.

Mes 6 — Nivel 5 (Jetpack Compose: fundamentos, layouts, Material 3)

Hito: Reconstrucción parcial de Diario de clase en Compose.

Métrica de progreso: Dominas state hoisting de forma demostrable.

Mes 7 — Nivel 5 cierre (Navigation Compose, side-effects, animaciones, responsive UI)

Hito: Diario de clase v2 (Compose) completo con navegación y tablet support.

Métrica de progreso: Checklist completo de Nivel 5.

Mes 8 — Nivel 6 (arquitectura: MVVM, Repository, UseCases)

Hito: Diario de clase v3 con capas domain/data/presentation.

Métrica de progreso: Puedes explicar la regla de dependencia de Clean Architecture.

Mes 9 — Nivel 6 cierre (Hilt, MVI, modularización básica)

Hito: Hilt integrado sin errores; versión MVI del proyecto como ejercicio.

Métrica de progreso: Checklist completo de Nivel 6 — ESTE ES EL PUNTO MEDIO REAL DEL ROADMAP.

Mes 10 — Nivel 7 (persistencia: Room, DataStore, offline-first)

Hito: Diario de clase con Room + relaciones + offline-first funcional.

Métrica de progreso: App funciona sin internet mostrando datos cacheados.

Mes 11 — Nivel 8 (networking: Retrofit, manejo de errores, auth)

Hito: Diario de clase conectado a API real simulada con manejo completo de estados de red.

Métrica de progreso: Los 4 estados de red (loading/éxito/error/sin conexión) manejados correctamente.

Mes 12 — Nivel 9 (testing: unit, MockK, Compose testing)

Hito: 70%+ de cobertura real en capa domain. CHECKPOINT: empieza a construir tu perfil freelance en paralelo (sin buscar clientes aún, solo preparar terreno).

Métrica de progreso: Puedes explicar mock vs fake vs stub sin ayuda.

Mes 13 — Nivel 10 (seguridad: OWASP, Keystore, R8, biometría)

Hito: Auditoría de seguridad completa del proyecto principal.

Métrica de progreso: Cero secretos en el historial de GitHub, verificado explícitamente.

Mes 14 — Nivel 11 (rendimiento: profiling, LeakCanary, Baseline Profiles)

Hito: Al menos un memory leak real encontrado y corregido.

Métrica de progreso: Mejora medible de startup time documentada.

Mes 15 — Nivel 12 (publicación: Play Console, Firebase, CI/CD)

Hito: Diario de clase publicado en Internal Testing con Crashlytics activo.

Métrica de progreso: Pipeline de CI corriendo tests automáticamente en cada push.

Mes 16 — Consolidación de portafolio (Fases 3-4): apps de nicho educativo, pulido general

Hito: Al menos 2 apps publicadas en Play Store (una debe ser la de nicho educativo).

Métrica de progreso: Portafolio de GitHub organizado y presentable, con README profesional en cada repo.

Mes 17 — Nivel 13 inicio (freelancing: perfil, propuestas, primeras cotizaciones)

Hito: Perfil completo en Workana y/o Upwork con portafolio enlazado.

Métrica de progreso: Al menos 5 propuestas reales enviadas a proyectos publicados.

Mes 18 — Freelancing activo — primeros proyectos reales pequeños

Hito: CHECKPOINT CLAVE: primer proyecto freelance cerrado (aunque sea de margen bajo).

Métrica de progreso: Primera reseña obtenida en la plataforma.

Mes 19-20 — Nivel 14 (Senior: ADRs, contribución open source, entrevistas simuladas) en paralelo con freelancing

Hito: Al menos una contribución real a un proyecto open source.

Métrica de progreso: 3 Architecture Decision Records escritos y defendibles en una conversación técnica.

Mes 21-24 — Consolidación: freelancing sostenido con reputación creciente, y — solo si lo decides — postulación a empleo formal como Android Developer

Hito: Ingresos freelance recurrentes O proceso de entrevistas formales activo, según lo que decidas priorizar en ese momento.

Métrica de progreso: Puedes resolver un problema de system design mobile básico en una simulación de entrevista sin bloquearte.

Criterio final: ¿cuándo estás listo?

Para empezar a buscar clientes freelance (aprox. mes 16-17): Portafolio con al menos 4-5 apps sólidas, 2 publicadas en Play Store, testing real presente, y la app de nicho educativo terminada.

Para postularte a un empleo formal como Android Developer (aprox. mes 20-24): Checklist completo de los 15 niveles, al menos una contribución open source, capacidad demostrada de defender decisiones de arquitectura en una entrevista simulada, y algoritmos/estructuras de datos practicados consistentemente en las últimas 8 semanas previas.